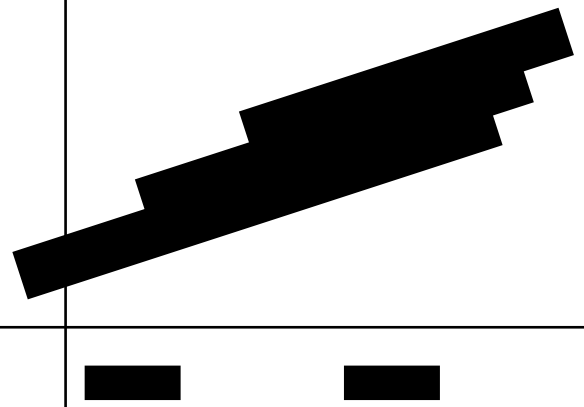
강서연 포트폴리오

SAP ABAP - Classic ABAP | S/4HANA RAP

paran312@naver.com



개발 프로그램 목록

	프로그램명	주요 내용	개발 시기	사용 기술
1	도서 자산관리 프로그램	도서 등록 / 대여 / 반납 , 사용자별 대여 상태 관리 , 엑셀 다운로드	2023	ABAP, ALV Grid, Selection screen, Excel Download
	창고 자산관리 프로그램	구매 물품 등록 , 물품 위치 및 수량 실시간 관리 , 재고 자동 계산	2023	ABAP, ALV Grid, F4 Search-help, Hotspot
2	자산 취득 요청 · 승인 관리 프로그램 (FI-AA / CO 마스터 참조)	자산클래스 , 비용센터 연동 취득 요청 작성 → 검증 → 승인 / 반려 프로세스	2026	RAP(Managed), CDS View, Composition, OData V4, Fiori Elements

* 2 번 (자산 취득 요청 · 승인 관리) 은 SAP RAP 학습을 반영한 프로젝트로 , 개발 서버에서 구현 및 화면 캡처 완료 후 상세 페이지를 이어서 정리했습니다 .

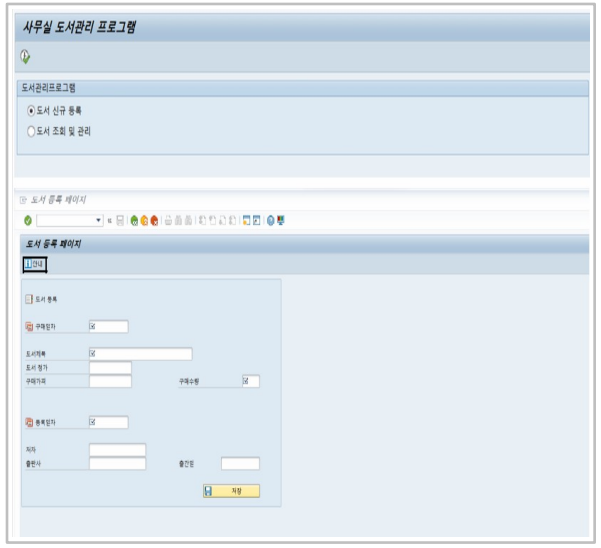
01 도서 / 창고 자산관리 프로그램 개요

01 개발 목적	AS-IS : 도서 / 비품 대여 수기 · 엑셀 관리 → 대여 추적 어려움 , 반납 누락 TO-BE : ALV 기반 관리 프로그램 → 물품 현황 실시간 조회 , 대여 이력 관리
02 프로그램 구성	**도서 자산 관리 프로그램** PC 자산 관리 프로그램 비품 관리 프로그램 **창고 자산 관리 프로그램**
03 개발 인원 / 기간	1 인 / 3 개월
04 담당 역할	**도서 자산 관리 프로그램** - 도서 등록 및 조회 기능 - 도서 대여 / 반납 프로세스 구현 - 사용자별 대여 상태 관리 기능 - 도서 정보 수정 및 관리 기능 구현 - 대여 도서 조회 및 엑셀 다운로드 기능 **창고 자산 관리 프로그램** - 구매 물품 등록 및 조회 기능 - 창고 위치 · 수량 관리 기능 구현 - 사용 수량 등록 및 재고 관리 기능 - 재주문 관리 기능 구현 - 물품 사용 이력 조회 기능

* 도서 / 창고 자산 관리 프로그램 (위 굵게 표시된 2 개) 은 본인이 요건분석부터 배포까지 단독으로 설계 · 개발했습니다 . (동일 프로젝트 내 PC· 비품 관리 프로그램은 별도 담당자가 개발)

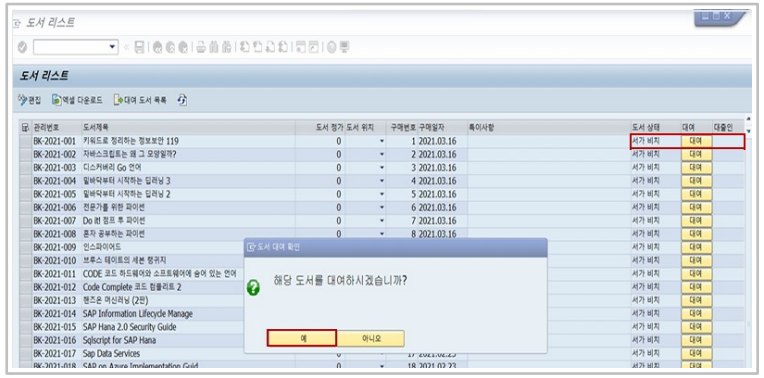
도서 등록 · 대여 · 반납

1 도서 등록



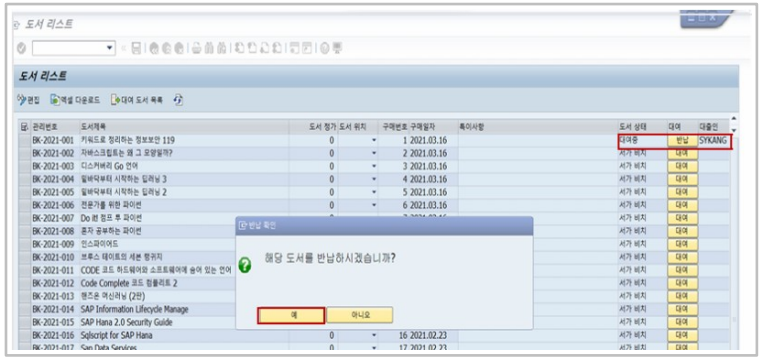
- 구매 도서 등록 기능 개발
- 필수 입력값 및 구매 / 등록일자 검증 로직 구현

2 도서 대여



- 대여 이력 생성 및 대여 상태 변경 기능 개발
- 도서 상태에 따른 버튼 동적 제어 구현

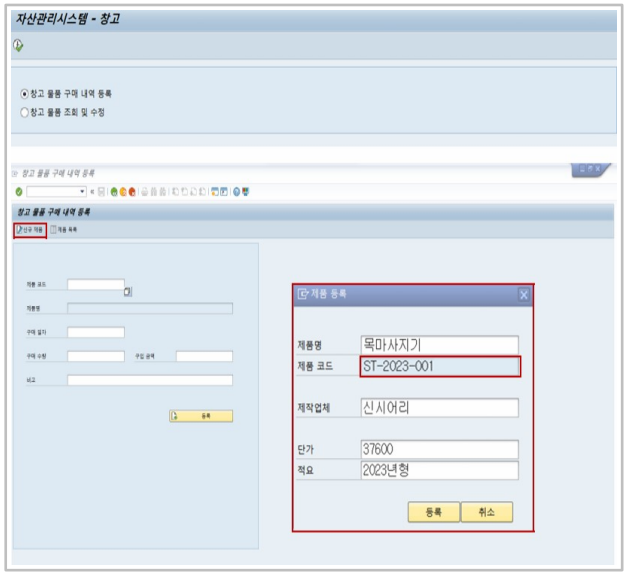
3 도서 반납



- 반납 이력 업데이트 , 대여정보 초기화 기능 개발
- 사용자 ID 기반 대여 / 반납 권한 제어 구현

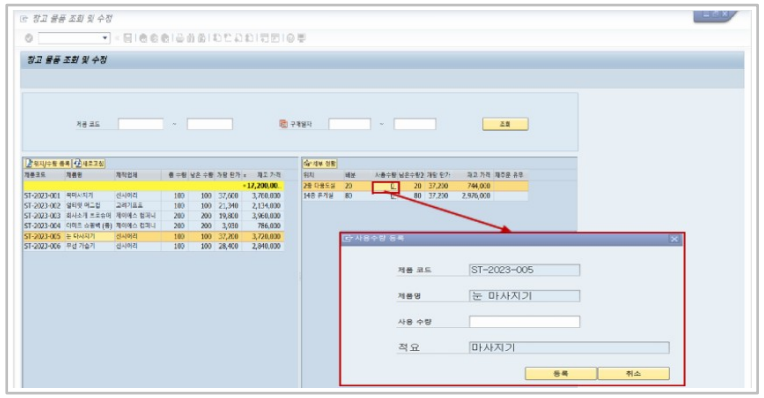
창고 물품 등록 · 조회 · 사용 이력

1 창고 물품 등록



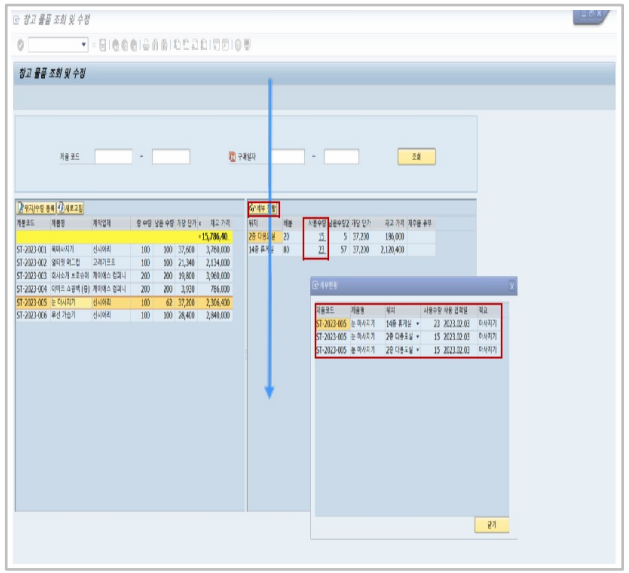
- 물품 등록 기능 개발
- 물품코드 자동 생성 및 금액 계산 구현
- Possible Entry(F4) 검색 기능 구현

2 위치 / 수량 관리



- 물품 선택 시 위치 / 수량 / 금액 상세 조회 기능 구현
- Hotspot 을 활용한 사용 수량 등록 기능 개발

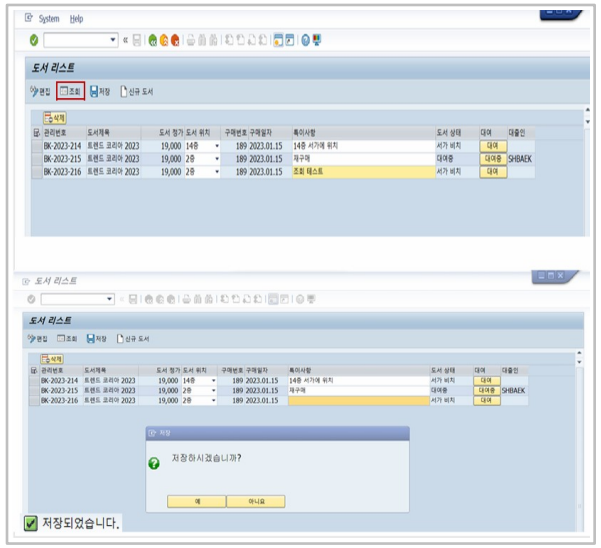
3 사용 이력 조회



- 물품 별 사용 수량 변경 이력 조회 기능 구현
- 최근 변경 내역 기준 사용 이력 관리 기능 개발

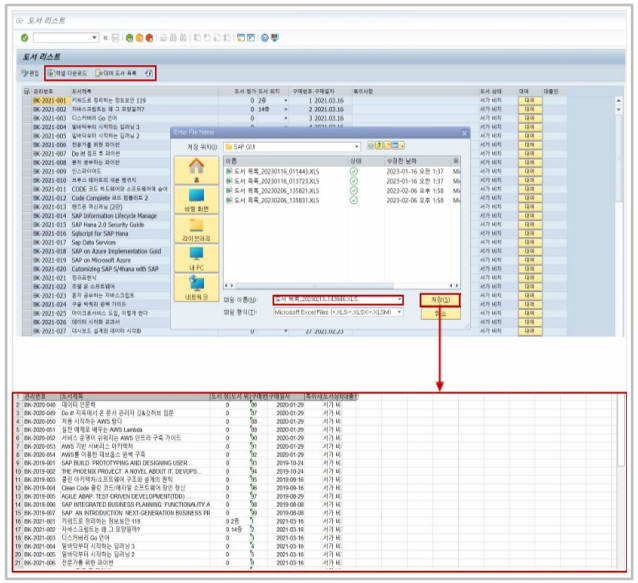
도서 관리 · Excel Download

1 도서 관리 (편집 · 저장)



- 편집 모드 전환 기능 구현
- 저장 / 삭제 시 확인 팝업 처리 구현

2 Excel Download



- ALV 조회 결과를 엑셀 파일로 저장하는 기능 구현

3 대여 도서 목록 조회



- 대출번호 기준 대여 중인 도서 목록 Pop-up 조회 기능 구현

02

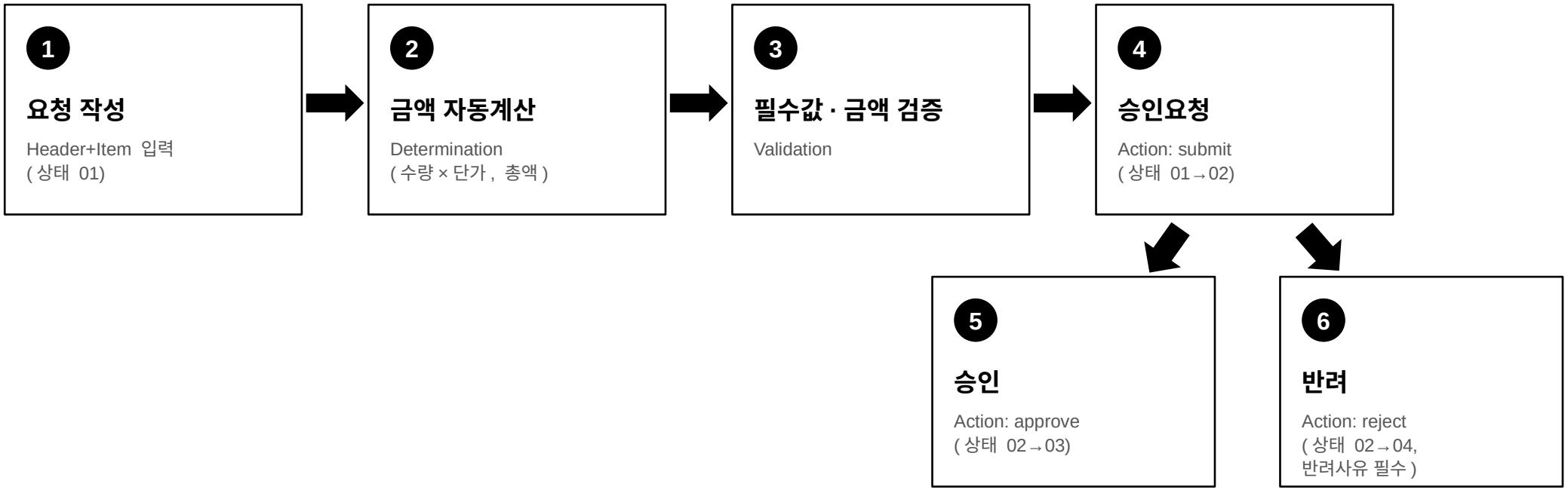
자산 취득 요청 , 승인 관리 프로그램 개요 (RAP)

01 개발 목적	AS-IS : 자산 취득 요청을 개별 확인 , 구두 승인으로 처리 → 이력 추적 어려움 , 승인 누락 위험 TO-BE : RAP 기반 승인 프로세스 프로그램 → 요청→검증→승인 / 반려 전 과정 시스템화
02 프로그램 구성	**자산 취득 요청 · 승인 관리 프로그램** FI-AA 자산클래스 및 CO 비용센터를 고려한 데이터 모델 설계
03 개발 인원 / 기간	1 인 주도 개발 (개인 학습 프로젝트) / SAP 개발 서버에서 설계부터 구현 및 테스트까지 직접 수행
04 담당 역할	- 자산 취득 요청 (Header/Item) 데이터 모델 설계 (CDS Composition, FI-AA/CO 마스터 참조) - Determination 구현 (요청번호 / 상태 자동세팅 , 금액 자동계산) - Validation 구현 (필수값 , 금액 조건 검증) - Action 구현 (승인요청 / 승인 / 반려 , 반려사유 필수 검증) - ABAP EML(MODIFY/READ ENTITIES) 기반 시나리오 테스트로 주요 로직 동작 검증

* FI-AA/CO " 참조 " 는 CDS 어소시에이션 ,F4 연결을 의미합니다 — 실제 자산마스터 / 전표 생성이나 BAPI 연동은 포함하지 않습니다 .

RAP 업무 프로세스

자산 취득 요청서 작성부터 승인 / 반려까지 , 상태 (Status) 기반으로 흐르는 업무 흐름입니다 .



핵심 구현

Determination

데이터를 자동으로 계산 · 세팅하는 로직

Validation

업무 규칙 위반을 저장 전에 막는 로직

Action

상태 전이로 업무 프로세스를 구현하는 로직

RAP 아키텍처 구조

데이터베이스 테이블부터 Fiori 화면까지 , RAP(Managed) 의 표준 레이어 구조를 그대로 따라 구현했습니다 .

1	**DB Table**	`ZTAA_REQ_H / ZTAA_REQ_I`	실제 데이터가 저장되는 물리 테이블
2	**CDS Interface View**	`ZI_AssetAcqRequest(Root) / ZI_AssetAcqRequestItem(Child)`	Composition 으로 헤더 - 아이템 연결 , FI-AA/CO 마스터 연관
3	**Behavior Definition (Managed)**	`Determination · Validation · Action` 정의	비즈니스 로직의 선언 및 구현 클래스 연결
4	**CDS Projection View**	`ZC_AssetAcqRequest / ZC_AssetAcqRequestItem`	UI 어노테이션 부여
5	**Service Definition / Binding**	`ZSD_ASSETACQREQUEST / ZSB_ASSETACQREQUEST (OData V4 - UI)`	프로젝션 뷰를 OData 서비스로 노출
6	**Fiori Elements**	`List Report + Object Page`	코드 없이 자동 생성되는 화면

구현 코드 화면

설계 (CDS/BDEF) 부터 실제 로직 구현까지 , ADT 에서 활성화 (초록 체크) 까지 완료한 코드입니다 .

1 CDS Interface View

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: '자산 취득 요청 (Interface)'
@Metadata.allowExtensions: true
define root view entity ZI_AssetAcqRequest
  as select from ztaa_req_h as Request

  composition [0..*] of ZI_AssetAcqRequestItem as _Item
{
  key request_id             as RequestId,
      request_no              as RequestNo,
      bukrs                   as Bukrs,
      req_date                as ReqDate,
      requester               as Requester,
      status                  as Status,
      reject_reason           as RejectReason,
      currency                as Currency,
      @Semantics.amount.currencyCode: 'Currency'
      total_amount            as TotalAmount,

      created_by              as CreatedBy,
      created_at              as CreatedAt,
      last_changed_by         as LastChangedBy,
      last_changed_at         as LastChangedAt,
      @Semantics.systemDateTime.lastChangedAt: true
      local_last_changed_at   as LocalLastChangedAt,

      case status
        when '04' then 1
        when '02' then 2
        else 3
      end as StatusCriticality,

      _Item
}
```

composition 으로 Header-Item 연결

2 Behavior Definition

```
managed implementation in class zbp_i_assetacqrequest unique;
strict ( 2 );

define behavior for ZI_AssetAcqRequest alias Request
persistent table ztaa_req_h
lock master
authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;

  determination setInitialData on modify { create; }
  determination calculateTotalAmount on modify { create; field Status; }

  field ( readonly )
    RequestNo, Status, TotalAmount,
    CreatedBy, CreatedAt, LastChangedBy, LastChangedAt, LocalLastChangedAt;

  field ( mandatory : create )
    Bukrs, ReqDate, Requester;

  mapping for ztaa_req_h
    {
      RequestId           = request_id;
      RequestNo           = request_no;
      Bukrs               = bukrs;
      ReqDate             = req_date;
      Requester           = requester;
      Status              = status;
      RejectReason        = reject_reason;
      Currency            = currency;
      TotalAmount         = total_amount;
      CreatedBy           = created_by;
      CreatedAt           = created_at;
      LastChangedBy       = last_changed_by;
      LastChangedAt       = last_changed_at;
      LocalLastChangedAt  = local_last_changed_at;
    }

  association _Item { create; }
  action submit result [1] $self;
  action approve result [1] $self;
  action reject parameter ZI_AssetAcqRequest_Reject result [1] $self;

}

define behavior for ZI_AssetAcqRequestItem alias Item
persistent table ztaa_req_i
lock dependent by _Request
authorization dependent by _Request
{
  update;
  delete;

  determination calculateAmount on modify { field Quantity, UnitPrice; }
   validation validateAmount on save { field Quantity, UnitPrice; }
```

Determination · Validation · Action 선언

3 Behavior Implementation (reject)

```
    READ ENTITIES OF ZI_AssetAcqRequest IN LOCAL MODE
      ENTITY Request
        ALL FIELDS WITH CORRESPONDING #( keys )
      RESULT DATA(updated_requests).

    result = VALUE #( FOR upd IN updated_requests
      ( %tky = upd-%tky %param = upd ) ).
  ENDMETHOD.

    METHOD reject.
    READ ENTITIES OF ZI_AssetAcqRequest IN LOCAL MODE
      ENTITY Request
        FIELDS ( Status )
        WITH CORRESPONDING #( keys )
      RESULT DATA(requests).

    LOOP AT keys INTO DATA(key).
      READ TABLE requests INTO DATA(request) WITH KEY %tky = key-%tky.

      IF request-Status <> '02'.
        APPEND VALUE #( %tky = key-%tky ) TO failed-request.
        APPEND VALUE #( %tky = key-%tky
                        %msg = new_message( id = 'ZTAA_MSG' number = '003'
                                            severity = if_abap_behv_message=>severity-error ) )
               TO reported-request.
        CONTINUE.
      ENDIF.

      IF key-%param-RejectReason IS INITIAL.
        APPEND VALUE #( %tky = key-%tky ) TO failed-request.
        APPEND VALUE #( %tky = key-%tky
                        %msg = new_message( id = 'ZTAA_MSG' number = '004'
                                            severity = if_abap_behv_message=>severity-error ) )
               TO reported-request.
        CONTINUE.
      ENDIF.

      MODIFY ENTITIES OF ZI_AssetAcqRequest IN LOCAL MODE
        ENTITY Request
          UPDATE FIELDS ( Status RejectReason )
          WITH VALUE #( ( %tky = key-%tky
                          Status = '04'
                          RejectReason = key-%param-RejectReason ) ).
    ENDLOOP.

    READ ENTITIES OF ZI_AssetAcqRequest IN LOCAL MODE
      ENTITY Request
        ALL FIELDS WITH CORRESPONDING #( keys )
      RESULT DATA(updated_requests).

    result = VALUE #( FOR upd IN updated_requests
      ( %tky = upd-%tky %param = upd ) ).
  ENDMETHOD.
```

반려 시 상태 (02 여부) 와 사유 입력 여부를 순차적으로 검증한 뒤 , 통과했을 때만 Status 를 04 로 갱신합니다 .

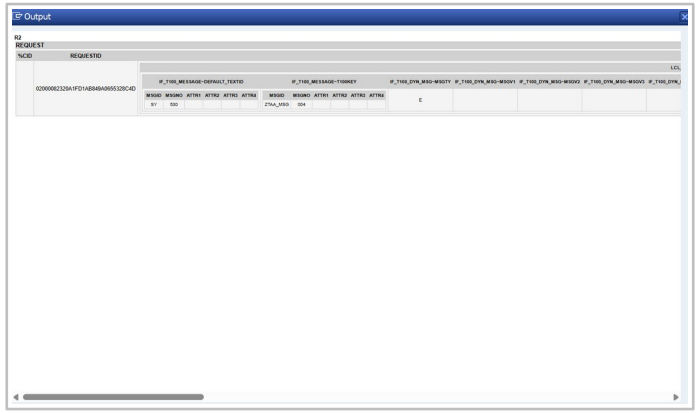
* 왼쪽 프로젝트 탐색기의 초록 체크 표시는 ADT 상 활성화 (빌드 성공) 상태를 의미합니다 .

반려 (Reject) 시나리오 실행 화면

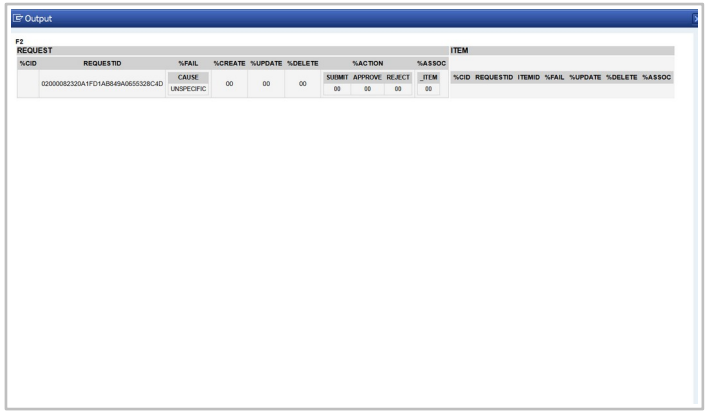
반려 사유 미입력 시 차단되고 , 사유 입력 시 정상 반려 처리되는 시나리오를 실제로 실행한 화면입니다 .

---

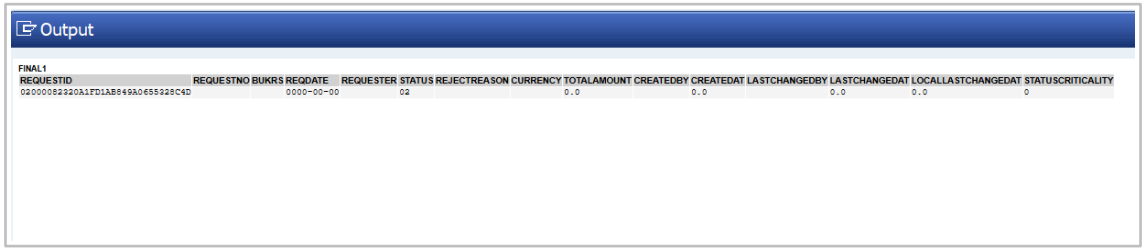
① 반려 사유 미입력 → 메시지 발생 (ZTAA_MSG 004)



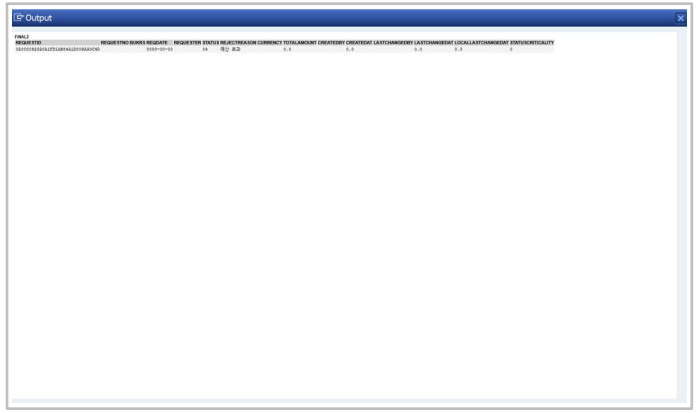
② 액션 실패 처리 확인 (%FAIL CAUSE: UNSPECIFIC)



③ 사유 없이 반려 시도 → Status 02 그대로 유지



④ 사유 입력 후 반려 성공 → Status 04, RejectReason 저장

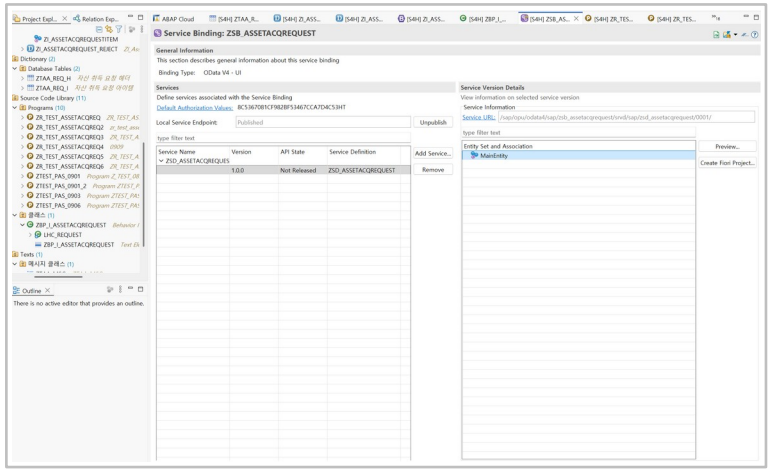


실행 화면

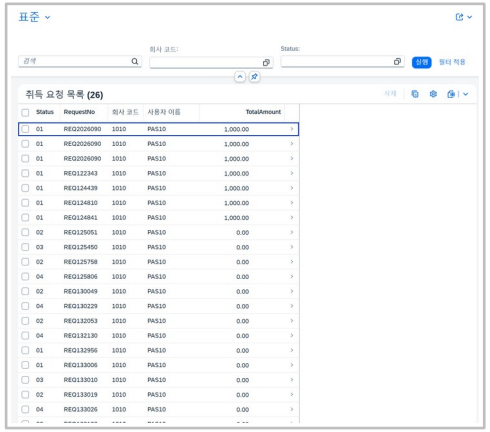
OData 서비스로 실제 발행되어 , Fiori 화면에서 데이터가 정상 조회되는 것까지 확인했습니다 .

---

1 Service Binding 발행 (Published) 상태 — ZSB_ASSETACQREQUEST (OData V4 - UI)



2 Fiori List Report — 필터 실행 후 실제 데이터 (RequestNo·Status·TotalAmount) 조회



* ADT Preview 환경에서는 Create 기능을 확인하지 못해 , CRUD 및 비즈니스 로직은 ABAP EML 테스트로 별도 검증했습니다 .

기존 프로젝트와의 비교

같은 SAP ABAP 개발자이지만 , 개발 방식과 접근이 어떻게 확장되었는지를 보여줍니다 .

01 도서 · 창고 자산관리	
개발 방식	Classic ABAP (SE80)
화면	ALV Grid, Selection Screen
데이터 처리	Open SQL 직접 제어
기능	Excel Download, 권한 제어 , 대여 / 반납 처리
역할	요건분석부터 테스트까지 단독 개발

02 자산 취득 요청 · 승인 관리	
개발 방식	ABAP Cloud / RAP (Managed)
화면	Fiori Elements
데이터 처리	CDS View + Composition (FI-AA·CO 마스터 참조)
기능	Determination·Validation·Action 기반 승인 프로세스
역할	설계부터 구현 · 테스트까지 단독 수행

* 기존 Classic ABAP 기반 개발 경험에서 CDS,Behavior Definition 중심의 RAP 모델 기반 개발까지 개발 영역을 확장했습니다 .

프로젝트 성과 및 배운점

프로젝트 성과

- 수기 및 엑셀 방식으로 관리되던 자산 관리 업무를 SAP 기반 프로그램으로 전환
- 사내 자산 관리 업무에 활용 가능한 수준의 프로그램 개발 및 실제 업무 적용
- SAP RAP(ABAP Cloud) 기반 승인 프로세스 프로그램으로 개발 영역 확장

경험

- 실제 사용자 요구사항을 분석하여 화면 , 데이터 구조 , 업무 프로세스 설계 경험
- 대여 / 반납 상태 관리 , 사용자 권한 제어 , 재고 자동 계산 로직 구현 경험
- ALV Grid, F4 Search-help, Hotspot 활용한 화면 개발 경험
- 요건분석부터 배포까지 두 프로그램 (도서 , 창고) 을 단독으로 설계 및 구현한 경험
- CDS/RAP 기반 Determination·Validation·Action 설계 및 구현 경험

실무 ABAP 개발 및 운영 경험과 RAP 기반 개발 경험을 바탕으로 업무 프로세스에 대한 이해를 넓혀
SAP 유지보수와 신규 개발을 함께 수행할 수 있는 개발자로 성장하겠습니다 .